Wayfork — Implementation Log

> Handoff document. If this session is lost, give this file (plus the original "Vario Product Specification & AI Engineering Blueprint") to any assistant to resume work. Prototype source: `wayfork-prototype.jsx`.

0. Naming decision

- Renamed **Vario → Wayfork** ("way" + "fork"): the core concept is a timeline that forks into alternative variants. Repo suggestion: `wayfork`.
- Rejected candidates: TripBranch, Splitinerary, Itinera.

1. Current state — Prototype v0.1

- Single-file React component (`wayfork-prototype.jsx`), default export `WayforkApp`.
- **In-memory only. No persistence, no backend, no build tooling yet.**
- Styling: Tailwind utility classes for layout + inline styles for the palette

(no Tailwind config/compiler assumed). No external dependencies beyond React.

2. Status matrix

✅ Implemented (working in prototype)

Spec section	Feature	Where
Domain model	TRIP / DAY / ITINERARY_SLOT / VARIANT_NODE / MICRO_STEP / EXPENSE_ITEM as factory functions	§1 of the .jsx
A. Ripple engine	Day start time editable; each slot start = previous slot end; recalculates on any change	`computeSchedule()`
A. Variants	2 slots with 2 variants each (OTP: public transit vs Uber; FCO: train+tram	`activeVariants` state

Spec section	Feature	Where
	vs taxi); toggle sets ACTIVE and ripples downstream	
A. Hard checkpoints	Flight boarding checkpoint (10:00, 20-min buffer); ok / amber / red states with computed margin	`CheckpointBanner`
B. Tri-currency	Home RON / Local EUR / Intl USD UI-wide toggle; EUR-pivot rate matrix cached as a constant; all conversion client-side	`RATES_EUR`, `convert()`
C. Ledger	Pre-trip vs mid-trip classification; payer attribution; native input	`MOCK_TRIP.expenses`

Spec section	Feature	Where
	currency preserved	
C. Splits	Split types: `equal`, `percent` (60/40 dinner demo), `fixed` (supported, not in mock data)	`expenseShares()`
C. Variant cost sync	Active variant costs summed and injected into "Projected total" card	`variantCostEUR`
C. Settlement	Net balances per participant + greedy minimal-transaction settlement list	`computeBalances()`, `settle()`

🟡 Partially implemented

- **Fixed-share splits**: engine supports `{type:'fixed'}` but no mock expense exercises it and there is no UI to author splits.

- **Variant cost → balances**: variant costs appear in the *projection* only.
  Design decision: unpaid projected costs are NOT netted into settlement
  (they have no payer yet). Spec was ambiguous here — revisit.
- **Micro-step model**: has `distanceKm` but distance is not rendered.
- **Weather state on VariantNode**: field exists in spec, **omitted** from the
  domain factories to keep v0.1 lean. Add `weather` to `VariantNode` next.
- **Multi-day**: model supports `trip.days[]` but UI renders `days[0]` only.

❌ Not implemented yet

- Persistence (DB / localStorage / API) — intentionally out of scope for v0.1.
- CRUD: adding/editing trips, slots, variants, steps, expenses (all data is mock).
- Live ECB rate fetch (rates are a hardcoded EUR-pivot constant).
- Timezones (OTP→FCO day is computed in one clock; real flight crosses TZ).
- Auth / multi-user sync / sharing.
- Checkpoint types other than time (e.g., opening hours).

- Tests.

3. Key algorithms & decisions (for reconstruction)

1. **Time** = integer minutes since midnight. `fmtTime/fmtDur` for display.
2. **Ripple**: `reduce` over slots; slot duration = Σ active variant micro-step durations; checkpoint `margin = checkpoint.time - slot.start`; status: `ok` if margin ≥ buffer, `amber` if 0 ≤ margin < buffer, `red` if late.
3. **Currency**: EUR-pivot matrix `{EUR:1, RON:4.97, USD:1.08}`; `convert(a,from,to) = a / rate[from] * rate[to]`. Fetched once at init in prod.
4. **Balances**: all expenses converted to EUR; payer credited full amount, every participant debited their share; greedy debtor↔creditor matching yields the minimal transaction list.
5. **Active variant** is UI state (`{slotId: variantId}`), not mutated into the data model — keeps mock data immutable and makes undo/URL-state trivial later.

4. Mock data snapshot

- Trip "Rome · May 2026", participants Andrei & Ioana, currencies RON/EUR/USD.
- Day 1 (2026-05-14), depart 06:30, 4 slots:
  1. Hotel → Otopeni — **fork**: Public transit (62m, 24 RON) vs Uber (39m, 95 RON)
  2. Check-in & security (55m)
  3. Flight OTP→FCO — **checkpoint** boarding 10:00, buffer 20m
  4. FCO → Trastevere — **fork**: Leonardo Express+tram (53m, 30 EUR) vs Taxi (53m… 8+45m, 55 EUR)
- Expenses: flights 1240 RON (Andrei, equal), apartment 380 EUR (Ioana, equal), dinner 62 EUR (Andrei, 60/40), metro 24 RON (Ioana, equal).

5. Suggested next steps (priority order)

1. Scaffold real repo (IntelliJ): Vite + React + TS. Split the single file into `domain/` (types + engines, framework-free), `data/mock.ts`, `ui/` components.
2. Convert factories to TypeScript interfaces; unit-test `computeSchedule`, `expenseShares`, `settle` (pure functions — easy wins).

3. Add CRUD forms + state management (Context or Zustand).
4. Persistence layer (start with localStorage adapter behind a repository
   interface, then swap for API).
5. ECB rate fetch with cached fallback; then weather per variant; then timezones.

6. How to resume with a fresh assistant

Paste the original Vario spec + this log, then say e.g.:
"Continue Wayfork from Prototype v0.1 as described in IMPLEMENTATION_LOG.md.
The single-file prototype is attached. Do step N of 'Suggested next steps'."
Update the Status matrix and this section after every working session.